Improbable Bigrams Expose Vulnerabilities of Incomplete Tokens in Byte-Level Tokenizers

Eugene Jang[1*] **Kimin Lee**[2] **Jin-Woo Chung**[3] **Keuntae Park**[3] **Seungwon Shin**[2]
[1]Northeastern University [2]KAIST [3]S2W Inc.
[1]jang.eu@northeastern.edu [2]{kiminlee,claude}@kaist.ac.kr
[3]{jwchung,keuntae.park}@s2w.inc

arXiv:2410.23684v2 [cs.CL] 10 Oct 2025

Abstract

Tokenization is a crucial step that bridges human-readable text with model-readable discrete tokens. However, recent studies have revealed that tokenizers can be exploited to elicit unwanted model behaviors. In this work, we investigate *incomplete tokens*, i.e., undecodable tokens with stray bytes resulting from byte-level byte-pair encoding (BPE) tokenization. We hypothesize that such tokens are heavily reliant on their adjacent tokens and are fragile when paired with unfamiliar tokens. To demonstrate this vulnerability, we introduce *improbable bigrams*: out-of-distribution combinations of incomplete tokens designed to exploit their dependency. Our experiments show that improbable bigrams are significantly prone to hallucinatory behaviors. Surprisingly, the same phrases have drastically lower rates of hallucination (90% reduction in Llama3.1) when an alternative tokenization is used. We caution against the potential vulnerabilities introduced by byte-level BPE tokenizers, which may introduce blind spots to language models.

1 Introduction

Tokenization is an important step in the large language model (LLM) pipeline, serving as the bridge between text inputs and the discrete tokens processed by the model. Improper tokenization can lead to undesirable behaviors, such as glitch token hallucinations (Rumbelow and Watkins, 2023; Land and Bartolo, 2024) and errors in numerical reasoning (Singh and Strouse, 2024). Furthermore, it has been observed that tokenizers can introduce elements of unfairness (Petrov et al., 2023) and bias (Ovalle et al., 2024) into models. Given the increasing duration and cost of model training, investigating and understanding the impact of the tokenizer is becoming increasingly important.

Recent works specifically analyze the tokenization step to identify inputs that provoke unwanted model behaviors. Land and Bartolo (2024) propose embedding layer heuristics to identify *glitch* tokens — undertrained tokens that cause hallucinations and also enable jailbreaks (Geiping et al., 2024). Wang et al. (2024) create adversarial questions designed to induce incorrect segmentation by the tokenizer, which degrades model performance.

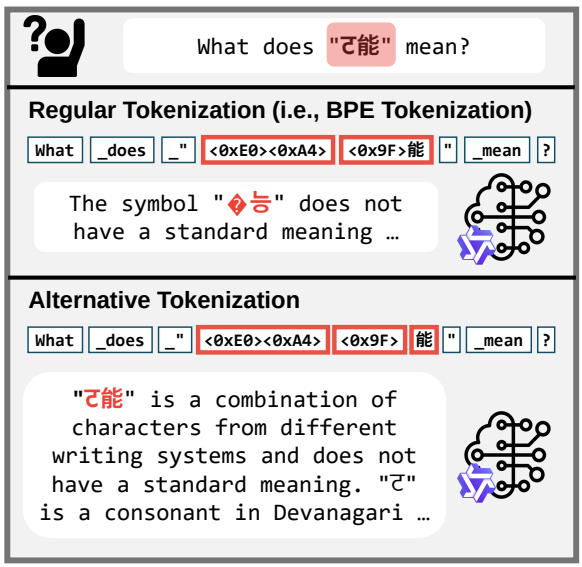

Figure 1: An improbable bigram phrase that combines two incomplete tokens to cause hallucinatory behaviors in the Qwen2.5 model. This behavior persists across multiple models and with well-trained tokens. An alternative tokenization of the same phrase does not cause hallucinations.

In this work, we investigate a specific vulnerability associated with *incomplete tokens* in byte-level byte pair encoding (BPE) tokenizers. These tokens, also known as undecodable tokens, are byte-level tokens that cannot be decoded independently and must appear in conjunction with certain other tokens to form legal Unicode characters. We explore the fragility of these tokens by constructing *improbable bigrams*, which are unlikely yet permissible combinations of two incomplete tokens. Similar to glitch tokens, improbable bigrams cause hallucinations to benign user requests, as seen in Figure 1.

[*] Work performed while at S2W Inc.

Tokenizer	Vocab Size	Incomplete Tokens	Incomplete Bigrams
Meta-Llama-3.1	128k	1224	71k
Exaone-3.0	102k	1222	36k
Qwen2.5	151k	1320	39k
Mistral-Nemo	131k	1307	135k
Command-R-v01	255k	2956	1479k

Table 1: Vocabulary sizes and number of incomplete tokens in different tokenizers. Incomplete bigrams refer to the total number of legal bigrams that can be created by combining two incomplete tokens.

However, unlike glitch tokens, improbable bigrams can cause hallucinatory behavior even when their constituent tokens are well-trained.

Our experiments across multiple LLM families (such as Llama-3.1, Qwen2.5, and Mistral-Nemo) demonstrate that these bigrams are significantly more prone to producing hallucinations compared to bigrams formed from complete tokens. Intriguingly, alternative tokenizations of the same phrases rarely exhibit hallucinations. We believe that our findings contribute to the growing body of research on understanding the potential weaknesses of BPE tokenization, with implications for the development of more robust language models.

2 BPE Tokenization

Byte pair encoding (BPE), originally developed as a data compression algorithm (Gage, 1994), has evolved into a popular tokenization scheme used by many modern language models (Sennrich et al., 2016). BPE tokenization iteratively expands the vocabulary by selecting frequent pairs of existing tokens as new vocabulary items. In particular, byte-level BPE, which applies the BPE algorithm at the byte-level rather than the character level (Wang et al., 2019), offers distinct advantages. Firstly, byte-level tokenization eliminates out-of-vocabulary issues by representing all Unicode characters (154,998 in Unicode v16.0) as combinations of 256 base bytes, ensuring comprehensive coverage. Secondly, it allows for more efficient data compression. The widespread adoption of byte-level tokenization in cutting-edge models such as GPT-4 and Llama 3.1 underscores its effectiveness.

While some have attributed BPE's effectiveness to its compression (Gallé, 2019; Goldman et al., 2024), recent work has challenged this assumption (Schmidt et al., 2024). Bostrom and Durrett (2020) point out that greedy compression prioritizes frequency over linguistically meaningful boundaries. This observation is supported by other works that report model improvements from morphologically motivated modifications to BPE tokenizers (Limisiewicz et al., 2024; Bauwens and Delobelle, 2024). These works demonstrate that incorrect segmentation by BPE tokenization cause suboptimal model performance.

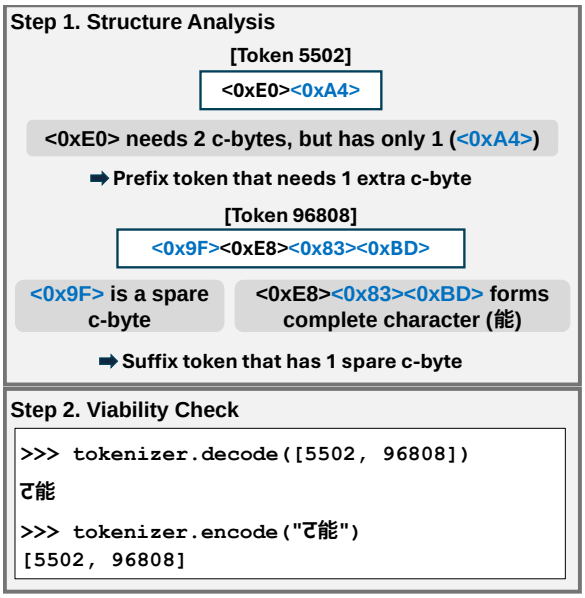

Figure 2: Incomplete tokens are analyzed for their structure to find potential bigram candidates. C-bytes (continuation bytes) are colored in blue. Tokens with compatible structures are tested for viability and reachability through an decode-encode check.

3 Incomplete Tokens and Improbable Bigrams

Byte-level BPE tokenizers are agnostic not only to morphemes but also to character boundaries. Training with the BPE algorithm on texts where characters are encoded with multiple bytes (e.g., Cyrillic (2 bytes), CJK Unified Ideographs (3 bytes)) often results in the formation of *incomplete tokens*. Incomplete tokens contain stray bytes (like <0x9F> in the token '<0x9F>能') that do not correspond to recognizable characters. These incomplete tokens constitute a small but significant portion of the vocabulary in byte-level models. Table 1 presents the number of discovered incomplete tokens in the vocabularies of various models employing byte-level tokenizers (see Appendix A for details).

While incomplete tokens are indistinguishable from complete tokens from the model's perspective

Task	User prompt
Repeat	`Repeat this phrase exactly: "{Phrase}"`
Definition	`What does "{Phrase}" mean?`
Do You Know?	`Today I heard about "{Phrase}". Do you know what this means?`
Usernames	`I am trying to learn python. What is the output of this? Print the output only with no further explanation.` `Usernames = ['LiquidBoxi胖胖', '{Phrase}', 'ɹəʞɹɐd', 'Jhocam(≠ `ɜ´)', 'AR皮球']` `print(Usernames[0:2])`

Table 2: Prompt templates used to test phrase-level hallucinations of target phrases. At test time, **{Phrase}** is replaced with the tested phrase. If the model fails to repeat the phrase in *all* four prompts, we consider the phrase to be hallucinatory.

during training, their structure can influence how they are trained in subtle ways. First, incomplete tokens can only occur alongside other incomplete or single-byte tokens, as the "stray bytes" of incomplete tokens require additional bytes to form a viable character. This restriction limits the range of tokens they can be associated with. Second, the potential characters that stray bytes can form into may have little to no semantic overlap, leading to a representation that is inherently ambiguous and heavily reliant on neighboring tokens for disambiguation. We hypothesize that these factors exacerbate one another, causing incomplete tokens to become overly dependent on their context and brittle in response to unfamiliar adjacent tokens.

To demonstrate this vulnerability, we create phrases that exploit incomplete tokens to cause hallucinations. First, we analyze the structure of incomplete tokens, categorizing those that end with stray bytes as *prefixes* and those that begin with stray bytes as *suffixes*. We further label prefixes and suffixes by the number of stray bytes they require to form a complete character, to find potential bigram combinations. We create adversarial combinations of these tokens, termed *improbable bigrams*, as illustrated in Figure 2. These bigrams exploit the dependency of two incomplete tokens to complete each other. We note that not all combinations are viable due to tokenization rules and merge priorities. Table 1 lists the number of all possible legal bigrams created from incomplete tokens across different tokenizers. To identify bigrams that are particularly out-of-distribution, we use the heuristic of multilinguality. When a bigram forms a string, we check the Unicode script type of each character. If the string contains characters from different Unicode scripts, we assume it is highly unlikely to have been encountered during training. In our experiments, we demonstrate that *improbable bigrams* are prone to hallucinations.

Models	Improbable Bigrams	Baseline Bigrams
Llama 3.1	48/100 (48%)	0/100 (0%)
Exaone	77/100 (77%)	20/100 (20%)
Qwen2.5	33/100 (33%)	0/100 (0%)
Mistral-Nemo	52/71 (73%)	1/71 (1%)
Command-R	49/100 (49%)	8/100 (8%)

Table 3: Number of hallucinations by **improbable bigram** phrases (with incomplete tokens) and by **baseline bigram** phrases (with complete tokens).

4 Hallucination Experiments

Models We experiment using five recently released instruction-trained models that use byte-level BPE tokenization: Meta-Llama-3.1-8B-Instruct[1], EXAONE-3.0.-7.8B-Instruct[2], Qwen2.5-32B-Instruct[3], Mistral-Nemo-Instruct-2407[4], and C4AI-Command-R-v01[5]. We use greedy decoding for all experiments.

Evaluation Prompts We assess each model's ability to accurately process prompts containing improbable bigram phrases. We use four prompt templates designed to induce the model to repeat a target phrase from the input, regardless of the phrase's nonsensical or unusual nature, as detailed in Table 2. Our goal is to find phrases that models cannot process for any prompt. Therefore, we

[1]https://huggingface.co/meta-llama/Meta-Llama-3.1-8B-Instruct

[2]https://huggingface.co/LGAI-EXAONE/EXAONE-3.0-7.8B-Instruct

[3]https://huggingface.co/Qwen/Qwen2.5-32B-Instruct

[4]https://huggingface.co/mistralai/Mistral-Nemo-Instruct-2407

[5]https://huggingface.co/CohereForAI/c4ai-command-r-v01